大数据应用案例

费子成　　苏州大学

E-mail: feizicheng@suda.edu.cn

个人主页：https://web.suda.edu.cn/feizicheng/





01

MapReduce编程实践

MapReduce编程实践



任务要求

文件A的内容如下：

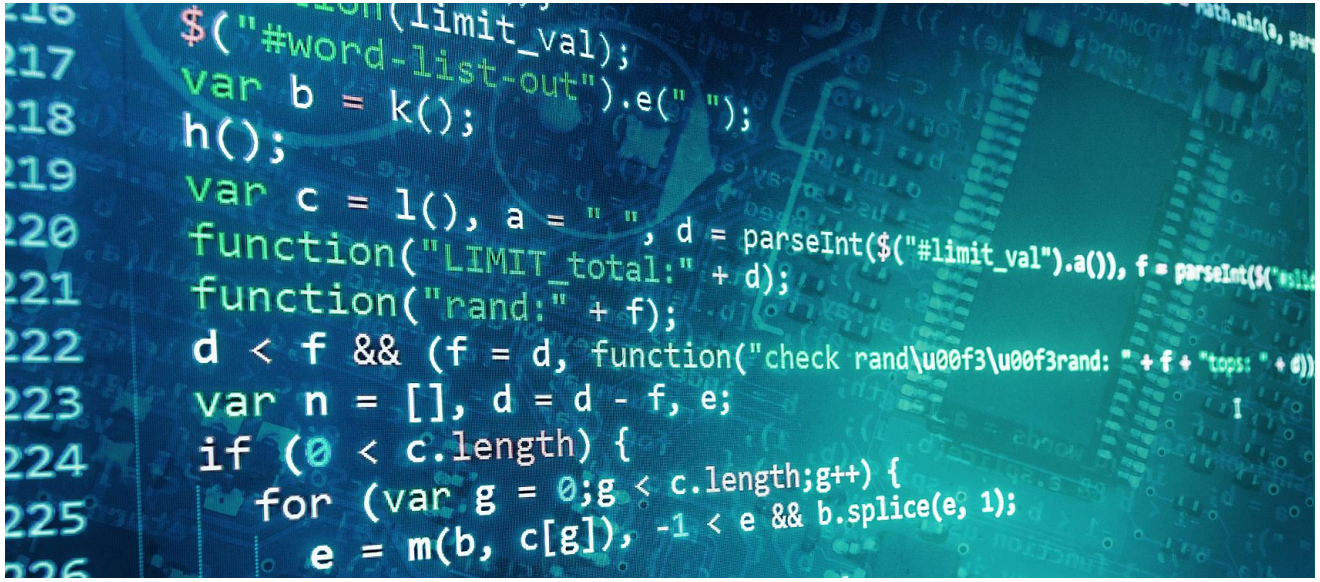
```
China is my motherland
I love China
```

文件B的内容如下：

```
I am from  China
```

期望结果如图所示：

```
I           2
is          1
China       3
my          1
love        1
am          1
from        1
motherland  1
```



编写Map处理逻辑

Map输入类型为<key,value>
期望的Map输出类型为<单词，出现次数>

Map输入类型最终确定为<Object,Text>
Map输出类型最终确定为<Text,IntWritable>

```
public static class MyMapper extends Mapper<Object,Text,Text,IntWritable>{
                    private final static IntWritable one = new IntWritable(1);
                    private Text word = new Text();
                    public void map(Object key, Text value, Context context) throws IOException,InterruptedException{
                        StringTokenizer itr = new StringTokenizer(value.toString());
                        while (itr.hasMoreTokens())
                        {
                            word.set(itr.nextToken());
                            context.write(word,one);
                        }
                    }
    }
```

编写Reduce处理逻辑

- 在Reduce处理数据之前，Map的结果首先通过Shuffle阶段进行整理。
- Reduce阶段的任务：对输入数字序列进行求和。
- Reduce的输入数据为<key,Iterable容器>。
- Reduce任务的输入数据：

```
<”I”,<1,1>>

<”is”,1>

……

<”from”,1>

<”China”,<1,1,1>>
```

编写Reduce处理逻辑

```
public static class MyReducer extends Reducer<Text,IntWritable,Text,IntWritable>{
                private IntWritable result = new IntWritable();
                public void reduce(Text key, Iterable<IntWritable> values, Context context)
throws IOException,InterruptedException{
                        int sum = 0;
                        for (IntWritable val : values)
                        {
                                sum += val.get();
                        }
                        result.set(sum);
                        context.write(key,result);
                }
        }
```

编写main方法

```
public static void main(String[] args) throws Exception{
        Configuration conf = new Configuration();  //程序运行时参数
        String[] otherArgs = new GenericOptionsParser(conf,args).getRemainingArgs();
        if (otherArgs.length != 2)
        {     System.err.println("Usage: wordcount <in> <out>");
              System.exit(2);
        }
        Job job = new Job(conf,"word count");  //设置环境参数
        job.setJarByClass(WordCount.class);  //设置整个程序的类名
        job.setMapperClass(MyMapper.class);  //添加MyMapper类
        job.setReducerClass(MyReducer.class);  //添加MyReducer类
        job.setOutputKeyClass(Text.class);  //设置输出类型
        job.setOutputValueClass(IntWritable.class);  //设置输出类型
        FileInputFormat.addInputPath(job,new Path(otherArgs[0]));  //设置输入文件
        FileOutputFormat.setOutputPath(job,new Path(otherArgs[1])); //设置输出文件
        System.exit(job.waitForCompletion(true)?0:1);
    }
```

完整代码

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.GenericOptionsParser;

public class WordCount{

//WordCount类的具体代码见下一页

}
```

完整代码

```
public class WordCount{
        public static class MyMapper extends Mapper<Object,Text,Text,IntWritable>{
            private final static IntWritable one = new IntWritable(1);
            private Text word = new Text();
            public void map(Object key, Text value, Context context) throws IOException,InterruptedException{
                StringTokenizer itr = new StringTokenizer(value.toString());
                while (itr.hasMoreTokens()){
                    word.set(itr.nextToken());
                    context.write(word,one);
                }
            }
        }
        public static class MyReducer extends Reducer<Text,IntWritable,Text,IntWritable>{
            private IntWritable result = new IntWritable();
            public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException,InterruptedException{
                int sum = 0;
                for (IntWritable val : values)
                {
                    sum += val.get();
                }
                result.set(sum);
                context.write(key,result);
            }
        }
        public static void main(String[] args) throws Exception{
            Configuration conf = new Configuration();
            String[] otherArgs = new GenericOptionsParser(conf,args).getRemainingArgs();
            if (otherArgs.length != 2)
            {
                System.err.println("Usage: wordcount <in> <out>");
                System.exit(2);
            }
            Job job = new Job(conf,"word count");
            job.setJarByClass(WordCount.class);
            job.setMapperClass(MyMapper.class);
            job.setReducerClass(MyReducer.class);
            job.setOutputKeyClass(Text.class);
            job.setOutputValueClass(IntWritable.class);
            FileInputFormat.addInputPath(job,new Path(otherArgs[0]));
            FileOutputFormat.setOutputPath(job,new Path(otherArgs[1]));
            System.exit(job.waitForCompletion(true)?0:1);
        }
```

编译打包代码以及运行程序

实验步骤：

01

使用java编译程序，生成.class文件

02

将.class文件打包为jar包

03

运行jar包（需要启动Hadoop）

04

查看结果



编译打包代码以及运行程序

- Hadoop 3.3.5 版本中的依赖 jar

使用 Hadoop 3.3.5 运行 WordCount 实例至少需要如下三个 jar:

```
$HADOOP_HOME/share/hadoop/common/hadoop-common-3.3.5.jar

$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-client-core-3.3.5.jar

$HADOOP_HOME/share/hadoop/common/lib/commons-cli-1.2.jar
```

通过命令 hadoop classpath 可以得到运行 Hadoop 程序所需的全部 classpath信息。

编译打包代码以及运行程序

将 Hadoop 的 classhpath 信息添加到 CLASSPATH 变量中，在 ~/.bashrc 中增加如下几行：

```
export HADOOP_HOME=/usr/local/hadoop export CLASSPATH=$($HADOOP_HOME/bin/hadoop classpath):$CLASSPATH
```

执行 source ~/.bashrc 使变量生效，接着就可以通过 javac 命令编译 WordCount.java

```
$javac WordCount.java
```

接着把 .class 文件打包成 jar，才能在 Hadoop 中运行：

```
jar -cvf WordCount.jar ./WordCount*.class
```

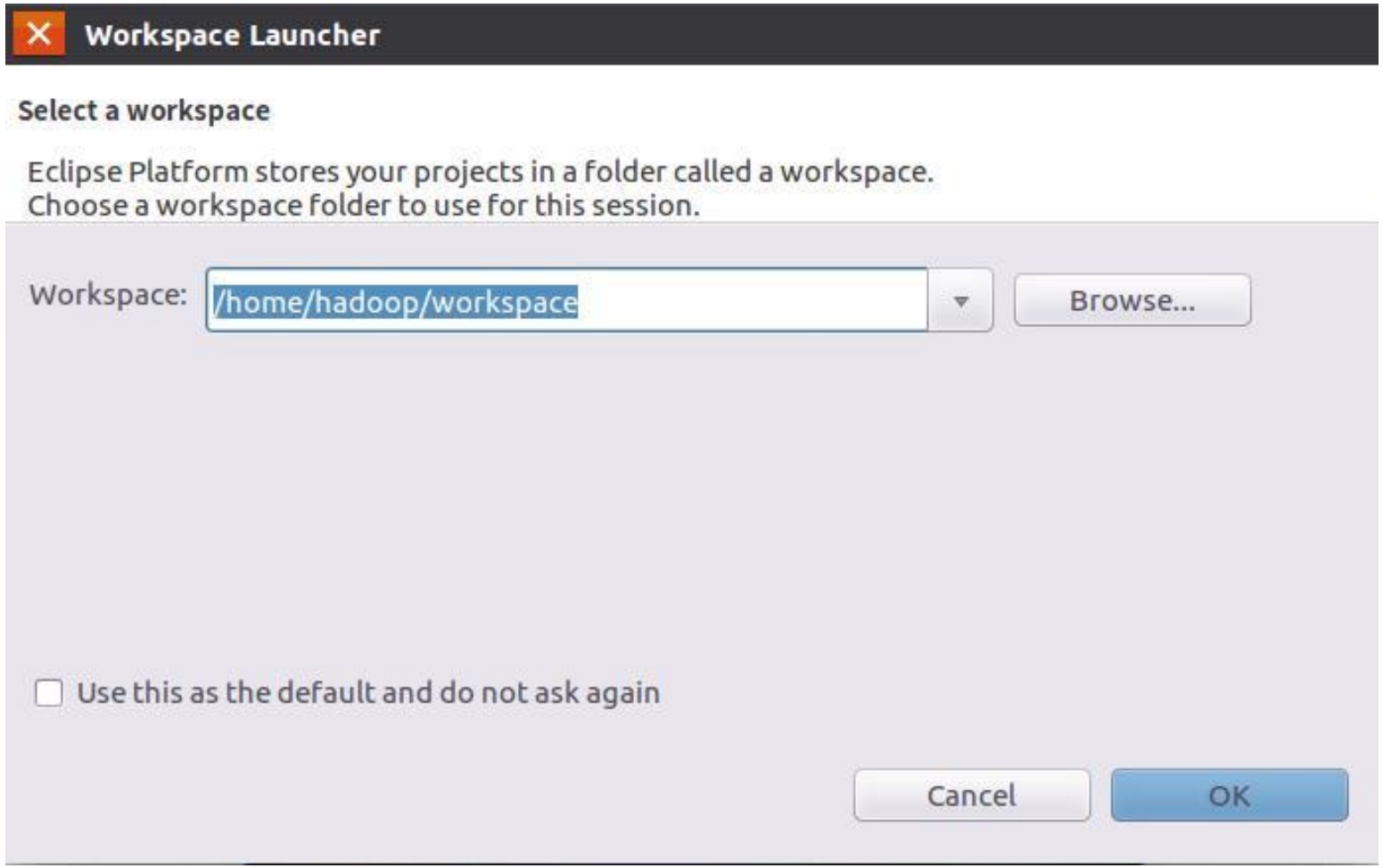
运行程序：

```
/usr/local/hadoop/bin/hadoop jar WordCount.jar WordCount input output
```

编译打包代码以及运行程序

- 在Eclipse中创建项目

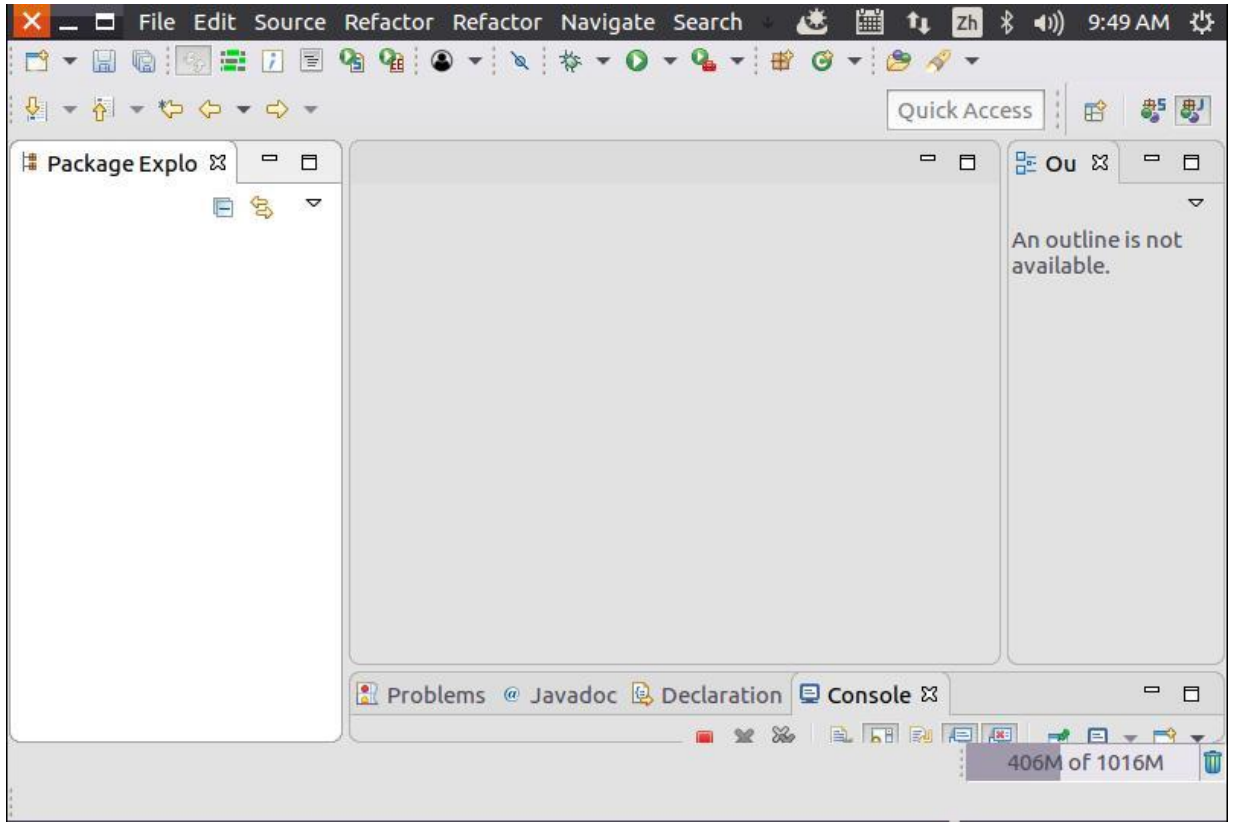
首先，启动Eclipse，启动以后会弹出如下图所示界面，提示设置工作空间（workspace）。



编译打包代码以及运行程序

- 在Eclipse中创建项目

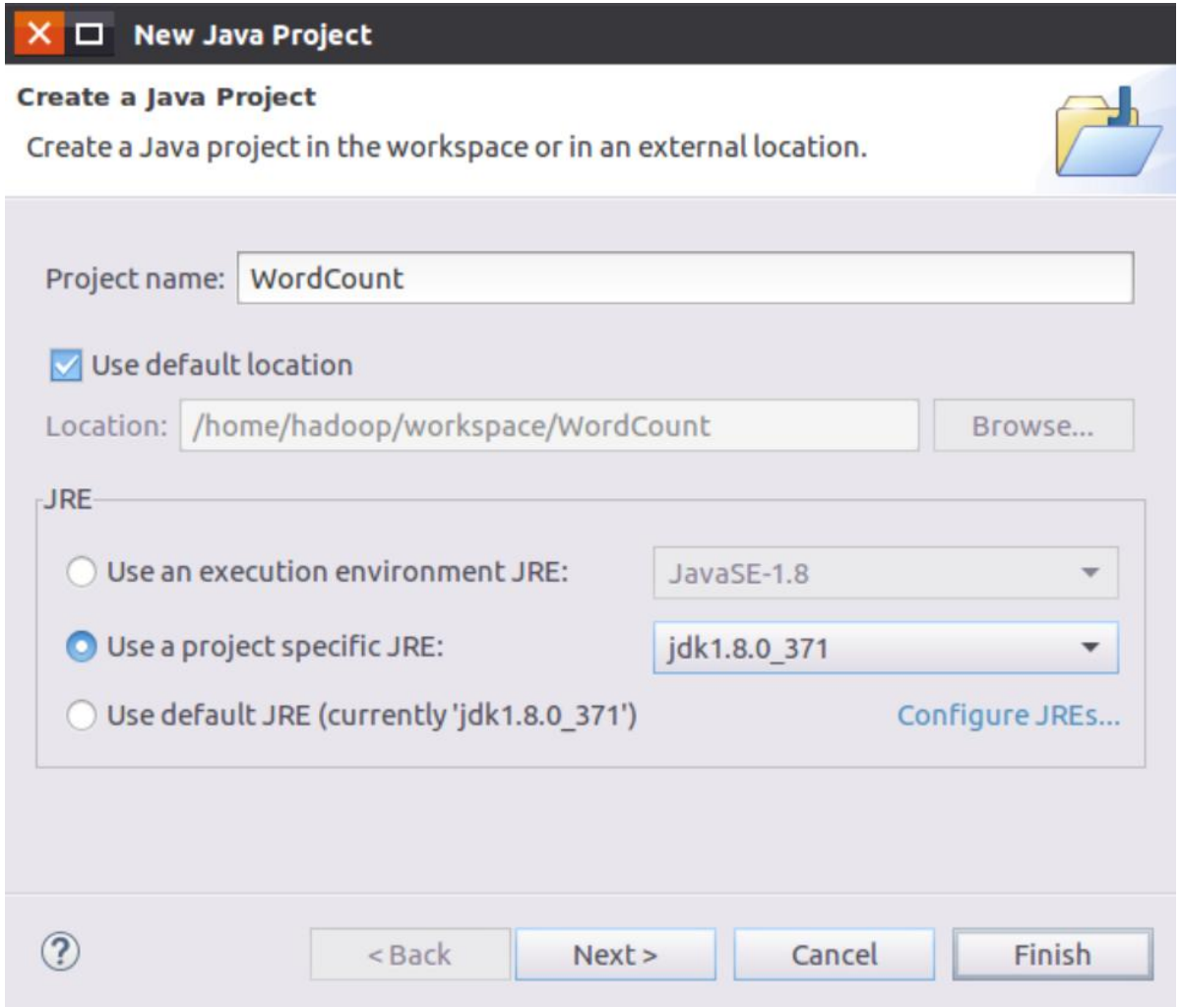
可以直接采用默认的设置“/home/hadoop/workspace”，点击“OK”按钮。可以看出，由于当前是采用hadoop用户登录了Linux系统，因此，默认的工作空间目录位于hadoop用户目录“/home/hadoop”下。Eclipse启动以后，呈现的界面如下图所示。



编译打包代码以及运行程序

- 在Eclipse中创建项目

选择“File–>New–>Java Project”菜单，开始创建一个Java工程，弹出如下图所示界面。



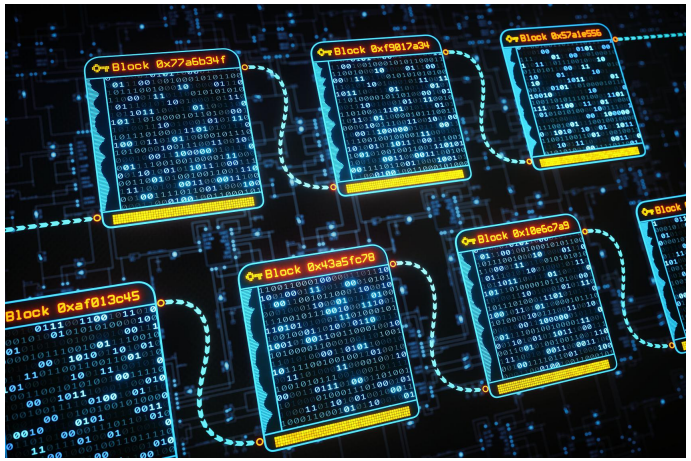
编译打包代码以及运行程序

- 在Eclipse中创建项目

在“Project name”后面输入工程名称“WordCount”，选中“Use default location”，让这个Java工程的所有文件都保存到“/home/hadoop/workspace/WordCount”目录下。

在“JRE”这个选项卡中，可以选择当前的Linux系统中已经安装好的JDK，比如jdk1.8.0_371。然后，点击界面底部的“Next>”按钮，进入下一步的设置。



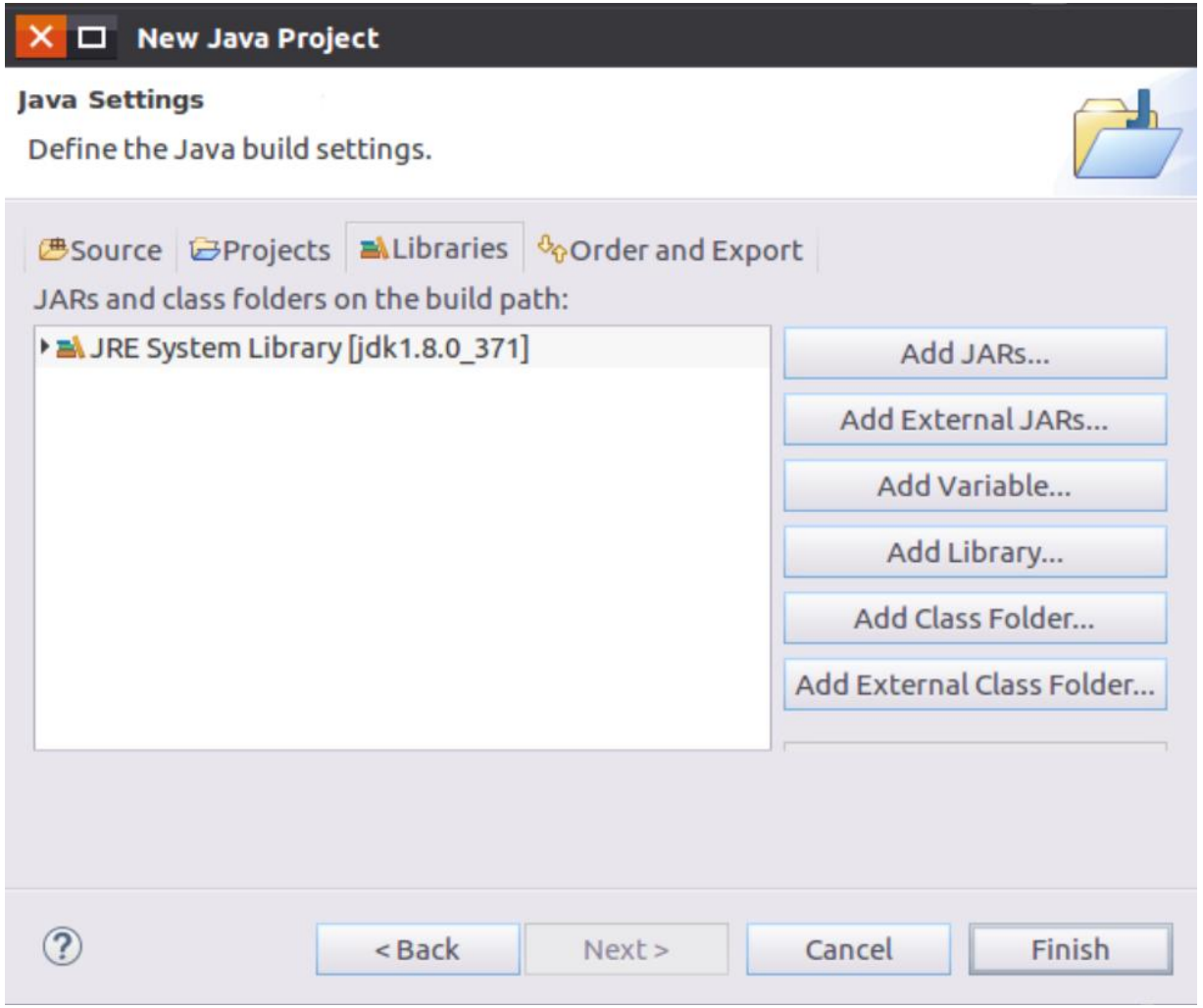




编译打包代码以及运行程序

- 为项目添加需要用到的JAR包

进入下一步的设置以后，会弹出如下图所示界面。

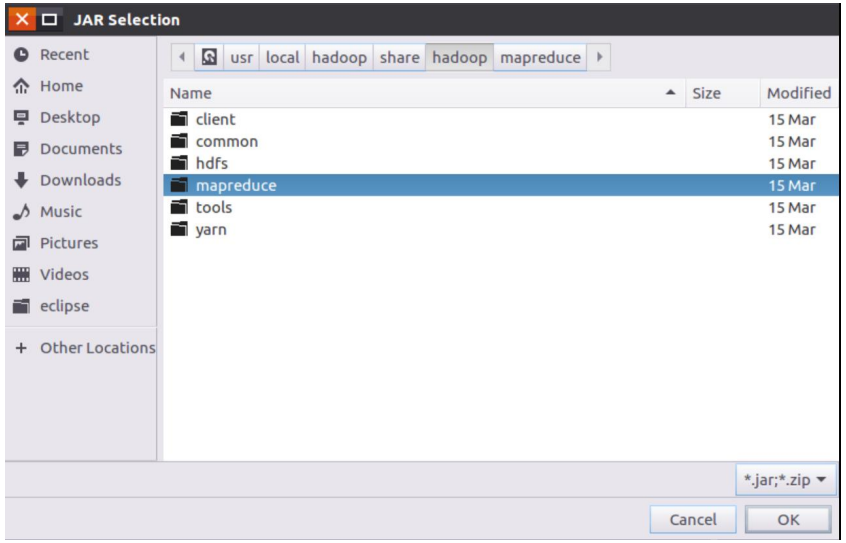


编译打包代码以及运行程序

- 为项目添加需要用到的JAR包

需要在这个界面中加载该Java工程所需要用到的JAR包，这些JAR包中包含了与Hadoop相关的Java API。这些JAR包都位于Linux系统的Hadoop安装目录下，对于本教程而言，就是在“/usr/local/hadoop/share/hadoop”目录下。

点击界面中的“Libraries”选项卡，然后，点击界面右侧的“Add External JARs...”按钮，弹出如下图所示界面。



编译打包代码以及运行程序

■ 为项目添加需要用到的JAR包

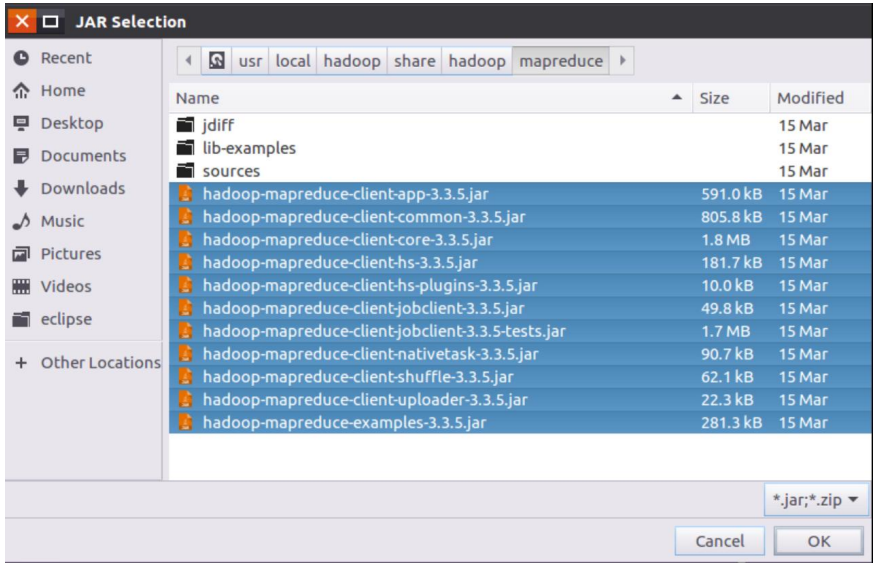
在该界面中，上面有一排目录按钮（即“usr”、“local”、“hadoop”、“share”、“hadoop”、“mapreduce”和“lib”），当点击某个目录按钮时，就会在下面列出该目录的内容。

为了编写一个MapReduce程序，一般需要向Java工程中添加以下JAR包：

（1）“/usr/local/hadoop/share/hadoop/common”目录下的hadoop-common-3.3.5.jar和haoop-nfs-3.3.5.jar；

（2）“/usr/local/hadoop/share/hadoop/common/lib”目录下的所有JAR包；

（3）“/usr/local/hadoop/share/hadoop/mapreduce”目录下的所有JAR包，但是，不包括jdiff、lib、lib-examples和sources目录，具体如下图所示。

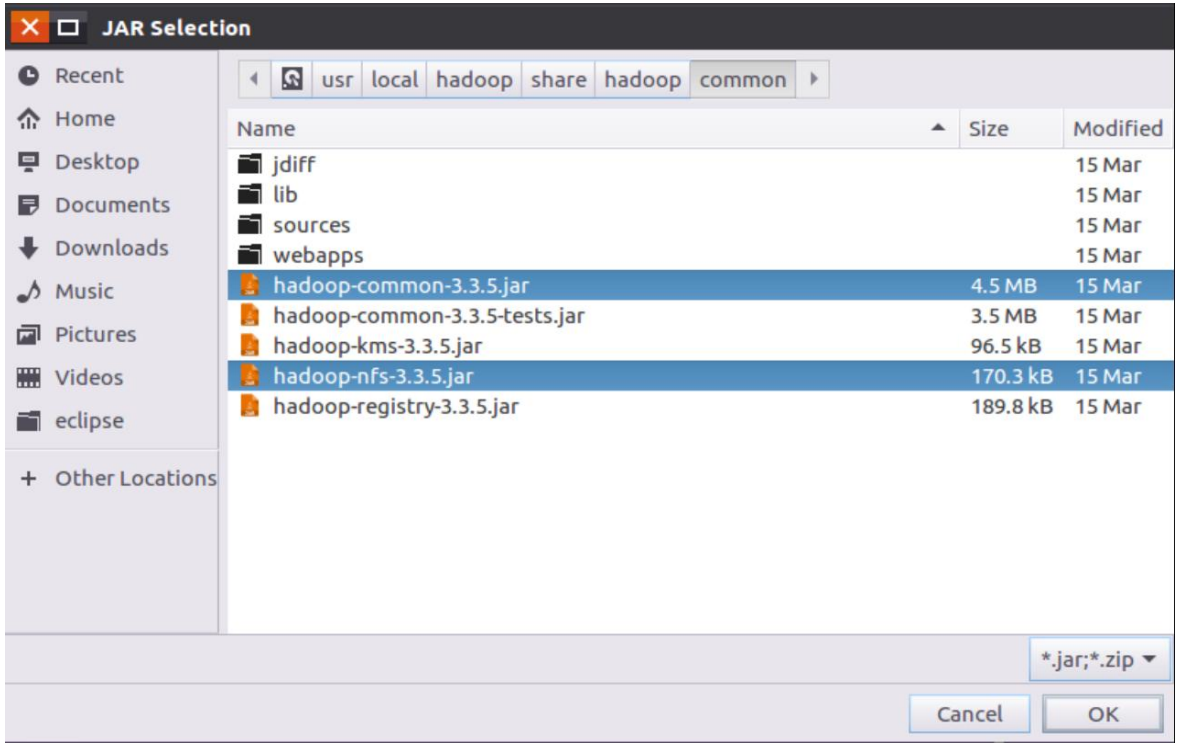


编译打包代码以及运行程序

- 为项目添加需要用到的JAR包

“/usr/local/hadoop/share/hadoop/mapreduce/lib”目录下的所有JAR包。

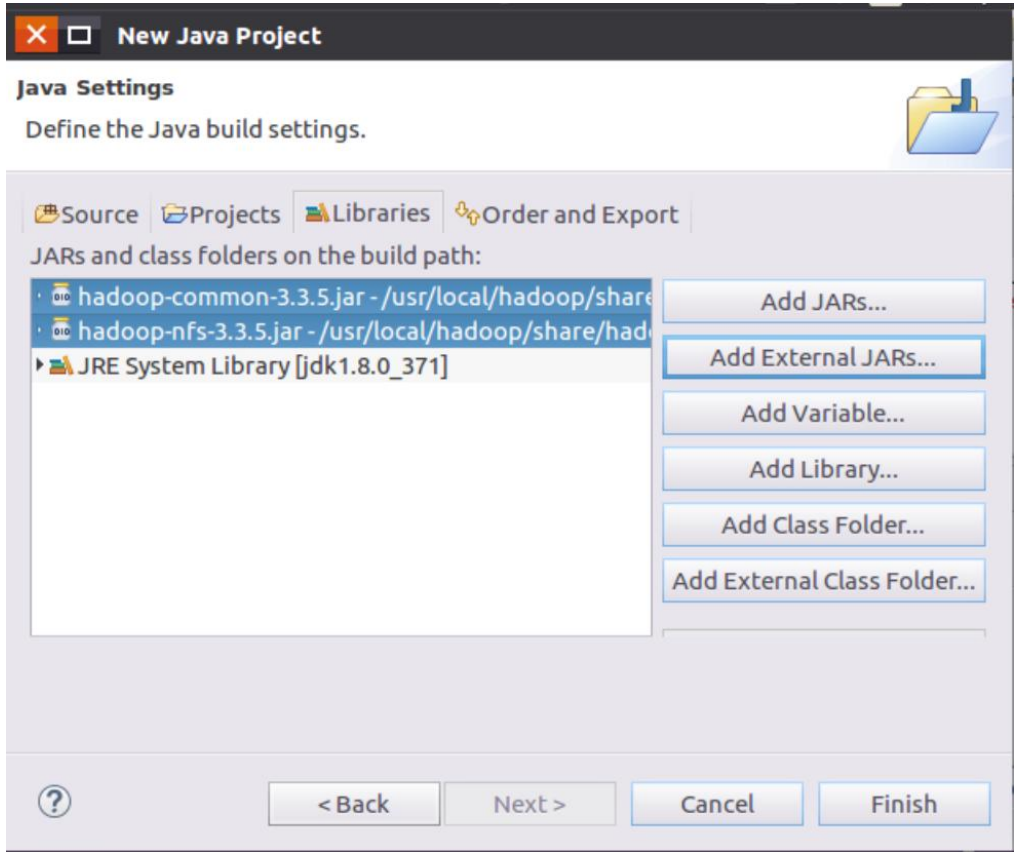
比如，如果要把“/usr/local/hadoop/share/hadoop/common”目录下的hadoop-common-3.3.5.jar和haoop-nfs-3.3.5.jar添加到当前的Java工程中，可以在界面中点击相应的目录按钮，进入到common目录，然后，界面会显示出common目录下的所有内容（如下图所示）。



编译打包代码以及运行程序

- 为项目添加需要用到的JAR包

请在界面中用鼠标点击选中hadoop-common-3.3.5.jar和haoop-nfs-3.3.5.jar，然后点击界面右下角的“确定”按钮，就可以把这两个JAR包增加到当前Java工程中，出现的界面如下图所示。

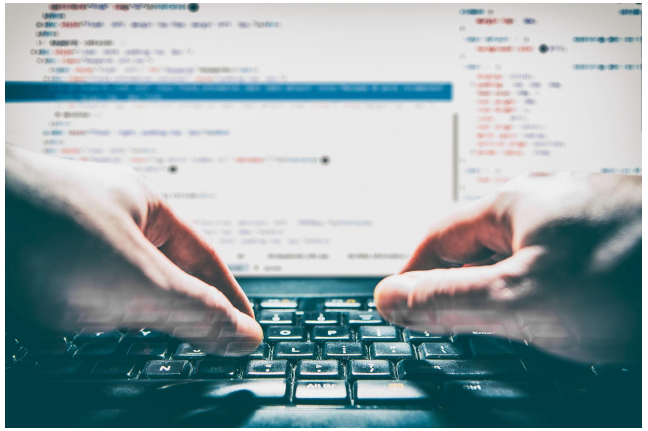


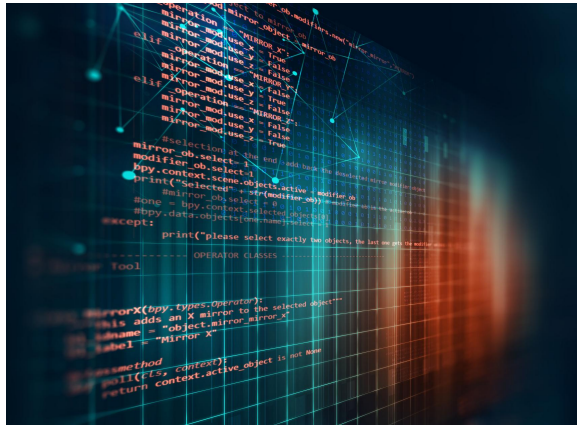
编译打包代码以及运行程序

- 为项目添加需要用到的JAR包

从这个界面中可以看出，hadoop-common-3.3.5.jar和haoop-nfs-3.3.5.jar已经被添加到当前Java工程中。然后，按照类似的操作方法，可以再次点击“Add External JARs...”按钮，把剩余的其他JAR包都添加进来。

需要注意的是，当需要选中某个目录下的所有JAR包时，可以使用“Ctrl+A”组合键进行全选操作。全部添加完毕以后，就可以点击界面右下角的“Finish”按钮，完成Java工程WordCount的创建。





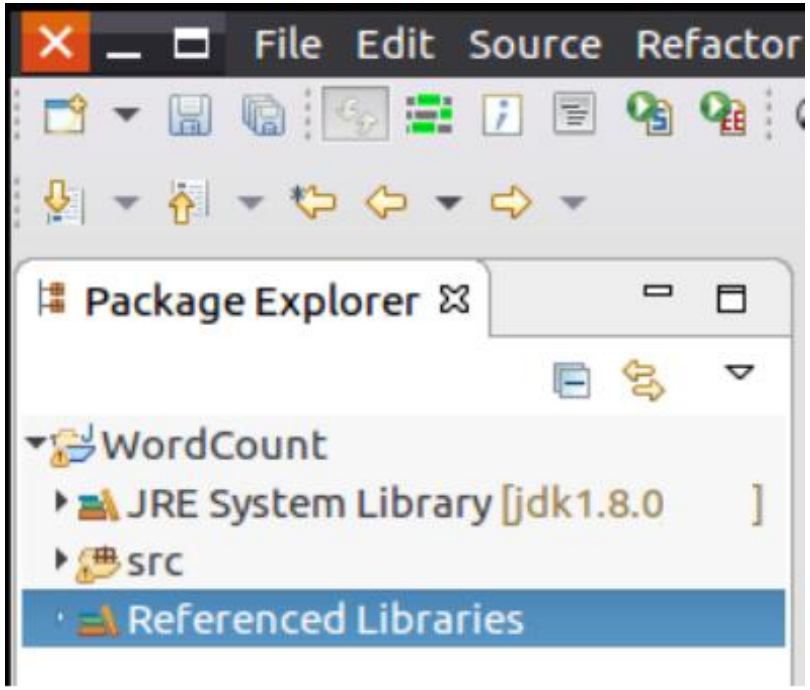


编译打包代码以及运行程序

■ 编写Java应用程序

下面编写一个Java应用程序，即WordCount.java。

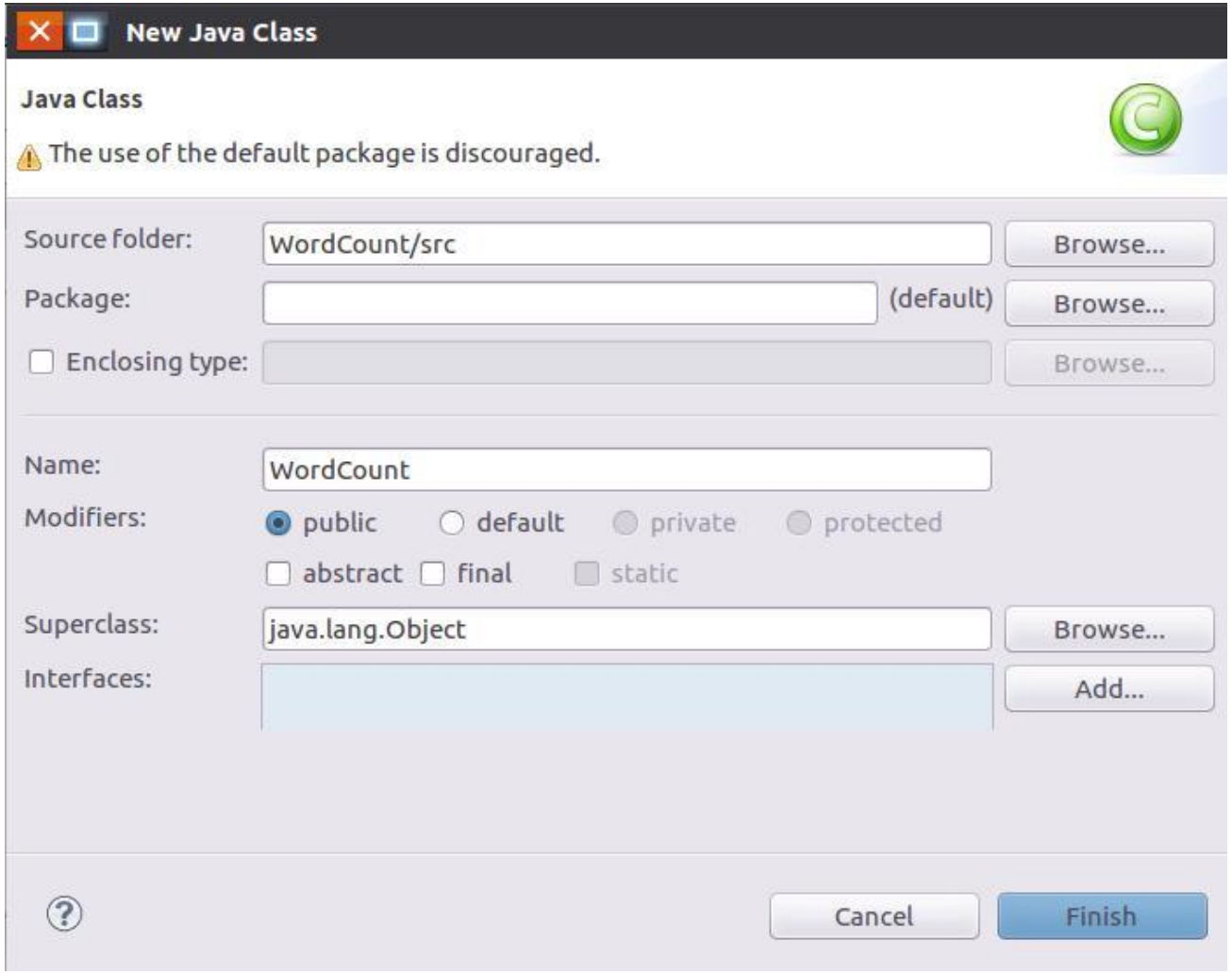
请在Eclipse工作界面左侧的“Package Explorer”面板中（如下图所示），找到刚才创建好的工程名称“WordCount”，然后在该工程名称上点击鼠标右键，在弹出的菜单中选择“New–>Class”菜单。



编译打包代码以及运行程序

- 编写Java应用程序

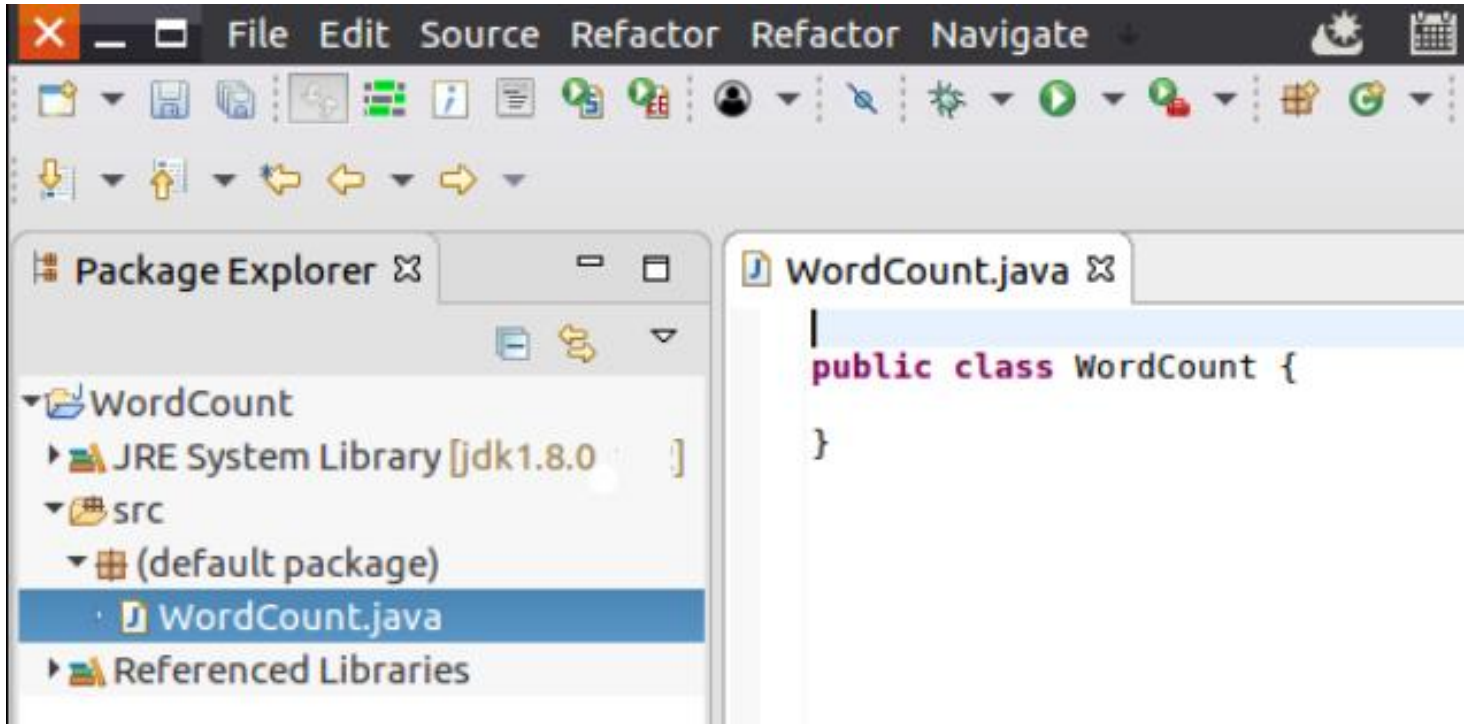
选择“New–>Class”菜单以后会出现如下图所示界面。



编译打包代码以及运行程序

- 编写Java应用程序

在该界面中，只需要在“Name”后面输入新建的Java类文件的名称，这里采用名称“WordCount”，其他都可以采用默认设置，然后，点击界面右下角“Finish”按钮，出现如下图所示界面。



编译打包代码以及运行程序

- 编写Java应用程序

可以看出，Eclipse自动创建了一个名为“WordCount.java”的源代码文件，并且包含了代码“public class WordCount{}”，请清空该文件里面的代码，然后在该文件中输入完整的词频统计程序代码，具体如下：

```
import java.io.IOException;
import java.util.Iterator;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.GenericOptionsParser;
public class WordCount {
    public WordCount() {
    }
```

```
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        String[] otherArgs = (new GenericOptionsParser(conf,
args)).getRemainingArgs();
        if(otherArgs.length < 2) {
            System.err.println("Usage: wordcount <in> [<in>...] <out>");
            System.exit(2);
        }
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(WordCount.TokenizerMapper.class);
        job.setCombinerClass(WordCount.IntSumReducer.class);
        job.setReducerClass(WordCount.IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        for(int i = 0; i < otherArgs.length - 1; ++i) {
            FileInputFormat.addInputPath(job, new Path(otherArgs[i]));
        }
```

编译打包代码以及运行程序

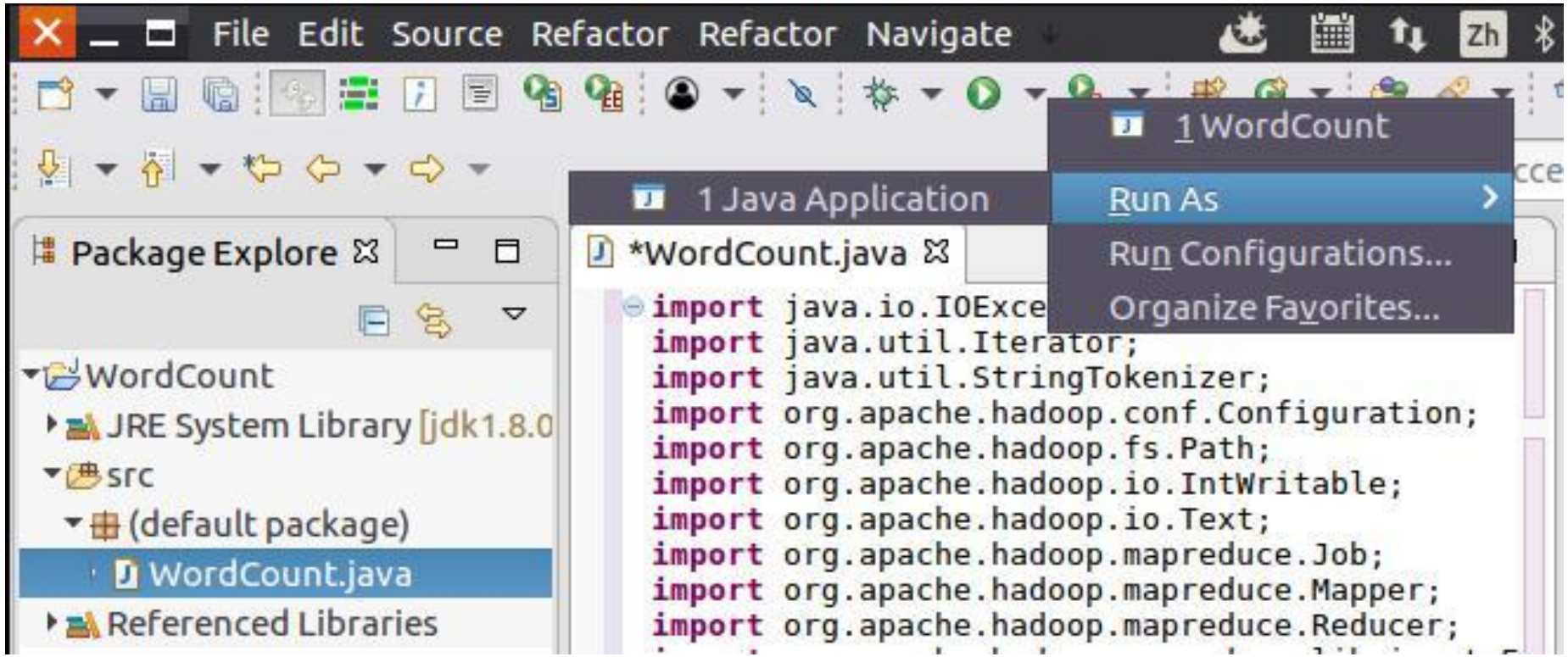
- 编写Java应用程序

```
FileOutputFormat.setOutputPath(job, new Path(otherArgs[otherArgs.length - 1]));
        System.exit(job.waitForCompletion(true)?0:1);
    }
    public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {
        private static final IntWritable one = new IntWritable(1);
        private Text word = new Text();
        public TokenizerMapper() {
        }
        public void map(Object key, Text value, Mapper<Object, Text, Text, IntWritable>.Context context) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while(itr.hasMoreTokens()) {
                this.word.set(itr.nextToken());
                context.write(this.word, one);
            }
        }
    }
public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
        private IntWritable result = new IntWritable();
        public IntSumReducer() {
        }
        public void reduce(Text key, Iterable<IntWritable> values, Reducer<Text, IntWritable, Text, IntWritable>.Context context) throws IOException,
InterruptedException {
            int sum = 0;
            IntWritable val;
            for(Iterator i$ = values.iterator(); i$.hasNext(); sum += val.get()) {
                val = (IntWritable)i$.next();
            }
            this.result.set(sum);
            context.write(key, this.result);
        }
    }
}
```

编译打包代码以及运行程序

■ 编译打包程序

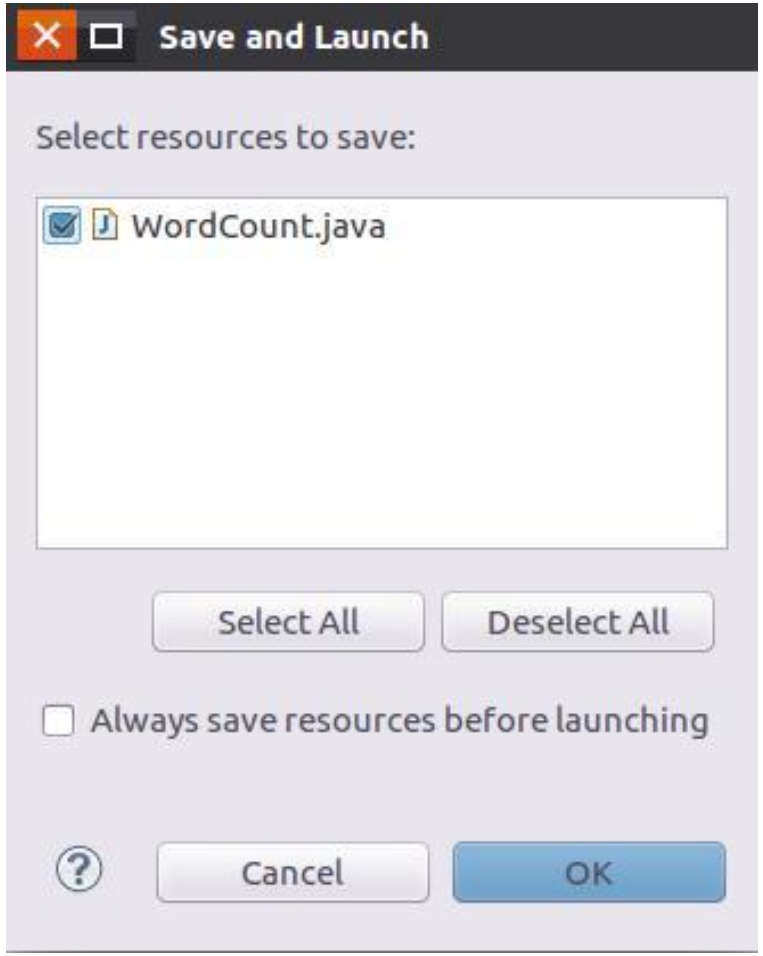
现在就可以编译上面编写的代码。可以直接点击Eclipse工作界面上部的运行程序的快捷按钮，当把鼠标移动到该按钮上时，在弹出的菜单中选择“Run as”，继续在弹出来的菜单中选择“Java Application”，如下图所示。



编译打包代码以及运行程序

- **编译打包程序**

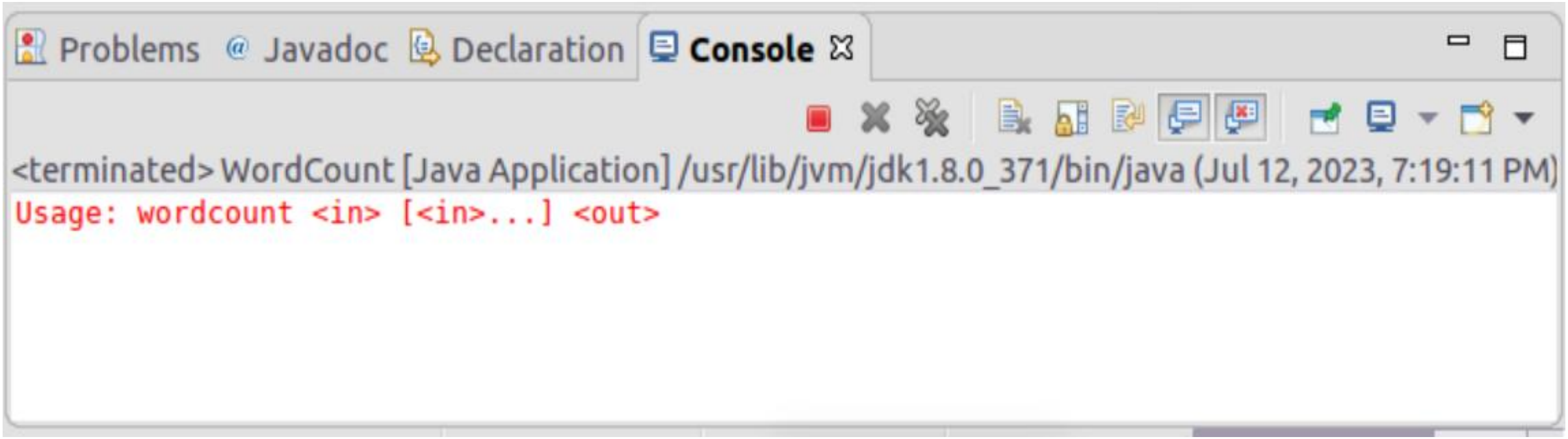
然后，会弹出如下图所示界面。



编译打包代码以及运行程序

- 编译打包程序

点击界面右下角的“OK”按钮，开始运行程序。程序运行结束后，会在底部的“Console”面板中显示运行结果信息（如下图所示）。



编译打包代码以及运行程序

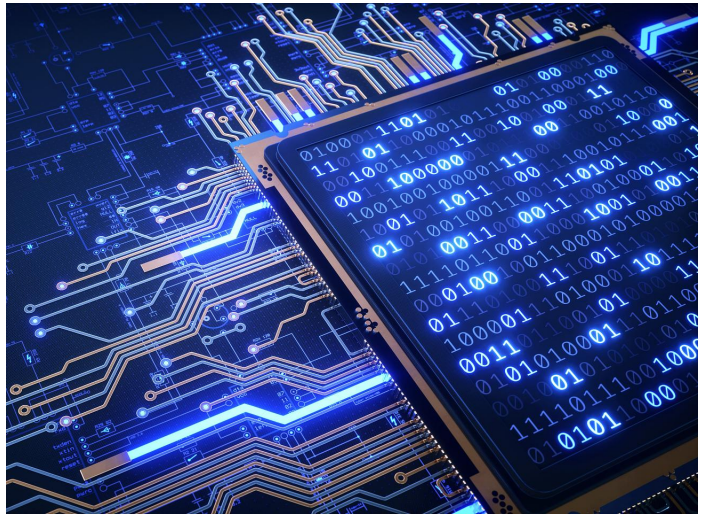
■ 编译打包程序

下面就可以把Java应用程序打包生成JAR包，部署到Hadoop平台上运行。现在可以把词频统计程序放在“/usr/local/hadoop/myapp”目录下。如果该目录不存在，可以使用如下命令创建：

```
$ cd /usr/local/hadoop
$ mkdir myapp
```



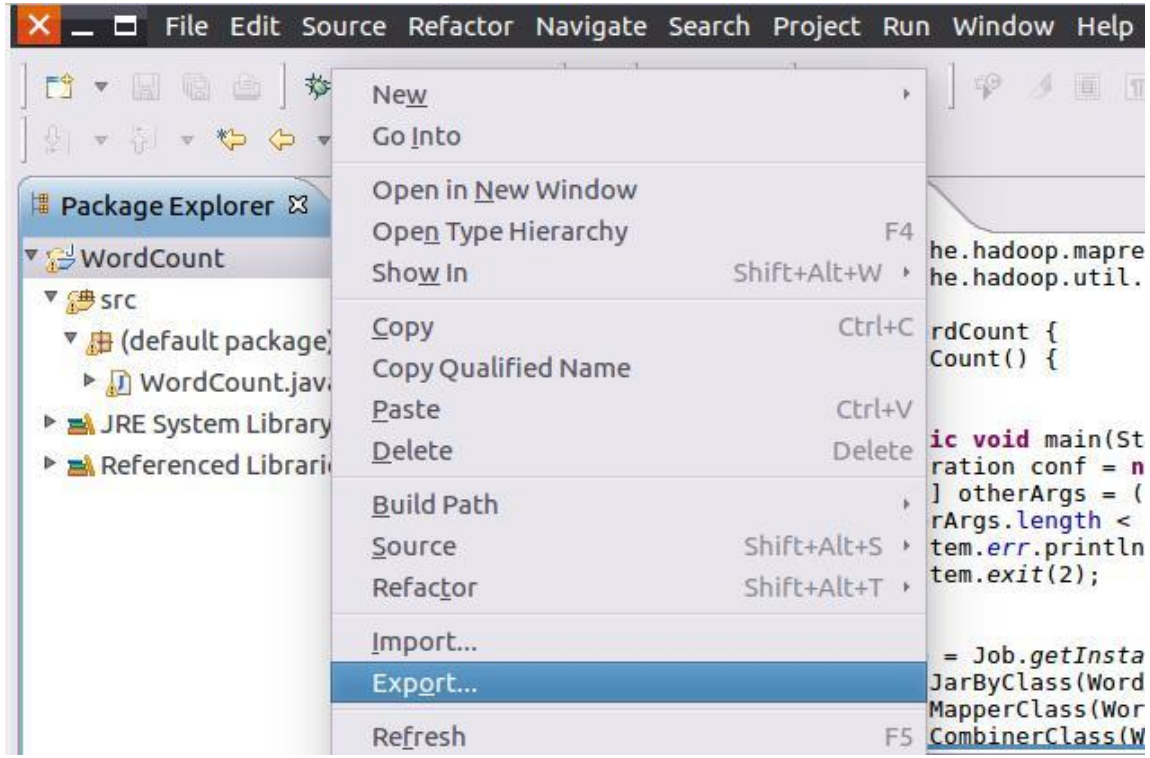




编译打包代码以及运行程序

■ 编译打包程序

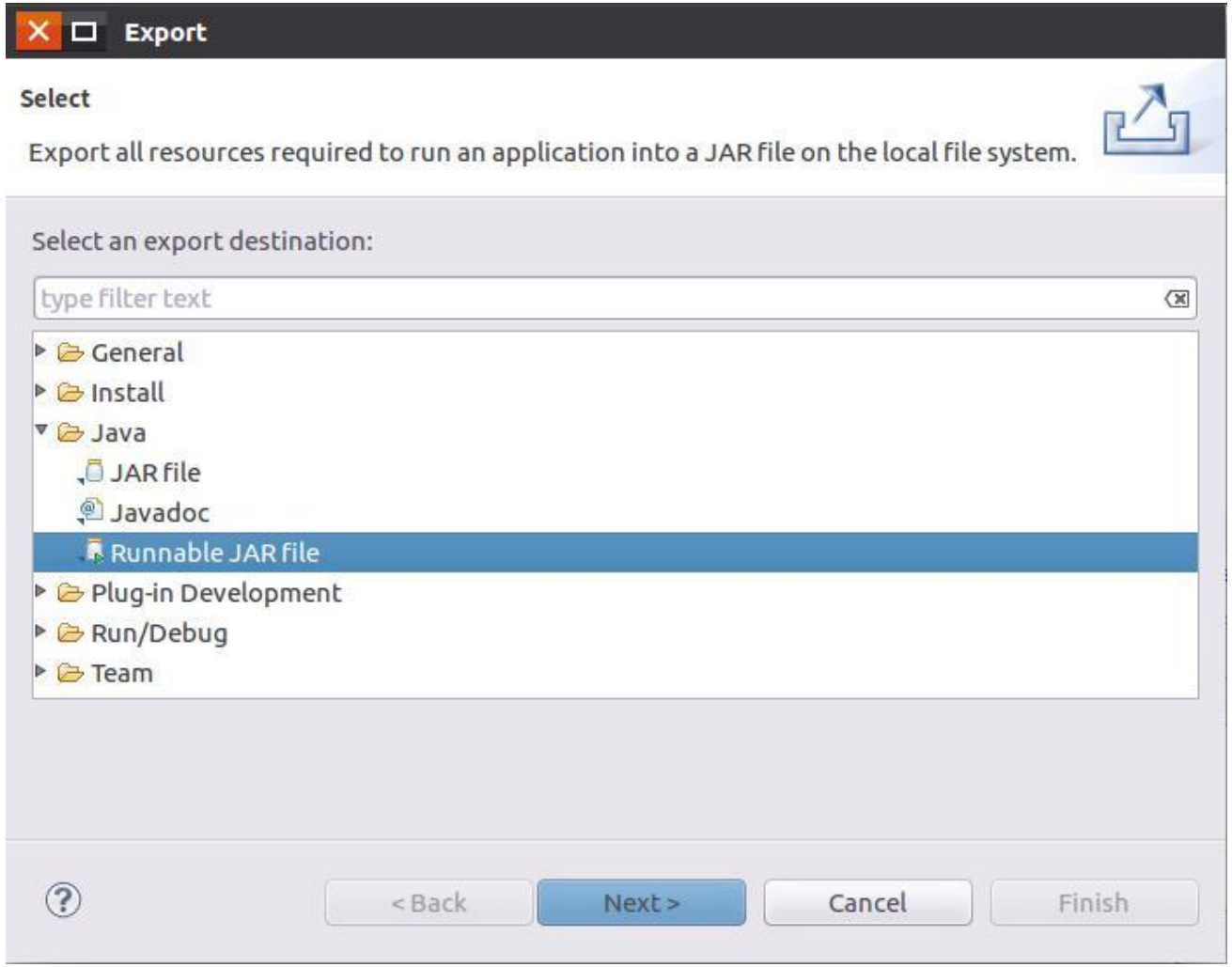
首先，请在Eclipse工作界面左侧的“Package Explorer”面板中，在工程名称“WordCount”上点击鼠标右键，在弹出的菜单中选择“Export”，如下图所示。



编译打包代码以及运行程序

- **编译打包程序**

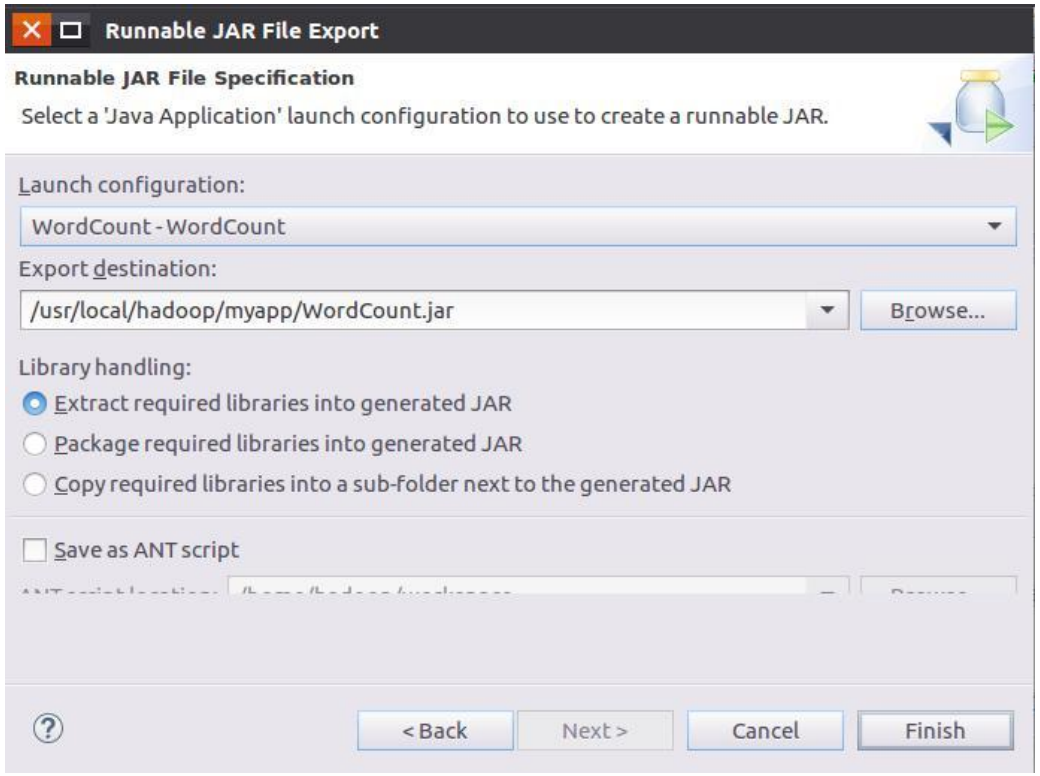
然后，会弹出如下图所示界面。



编译打包代码以及运行程序

■ 编译打包程序

在该界面中，选择“Runnable JAR file”，然后，点击“Next>”按钮，弹出如下图所示界面。

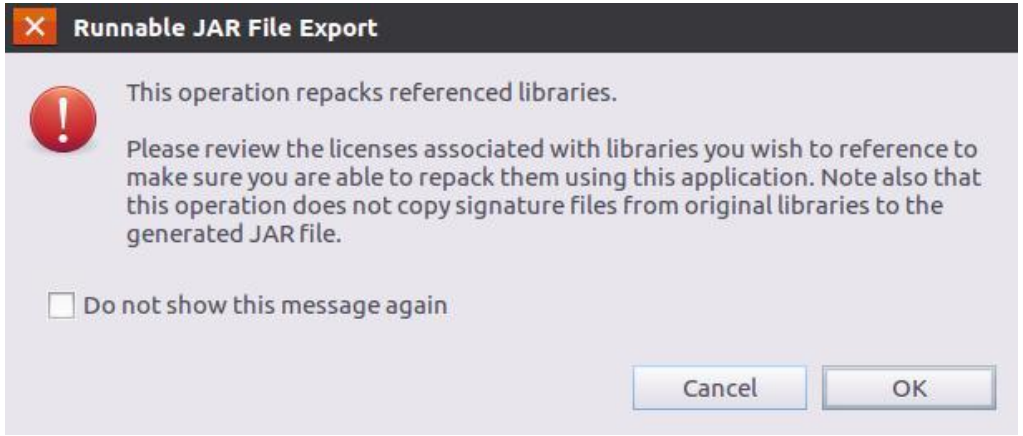


编译打包代码以及运行程序

■ 编译打包程序

在该界面中，“Launch configuration”用于设置生成的JAR包被部署启动时运行的主类，需要在下拉列表中选择刚才配置的类“WordCount-WordCount”。在“Export destination”中需要设置JAR包要输出保存到哪个目录。

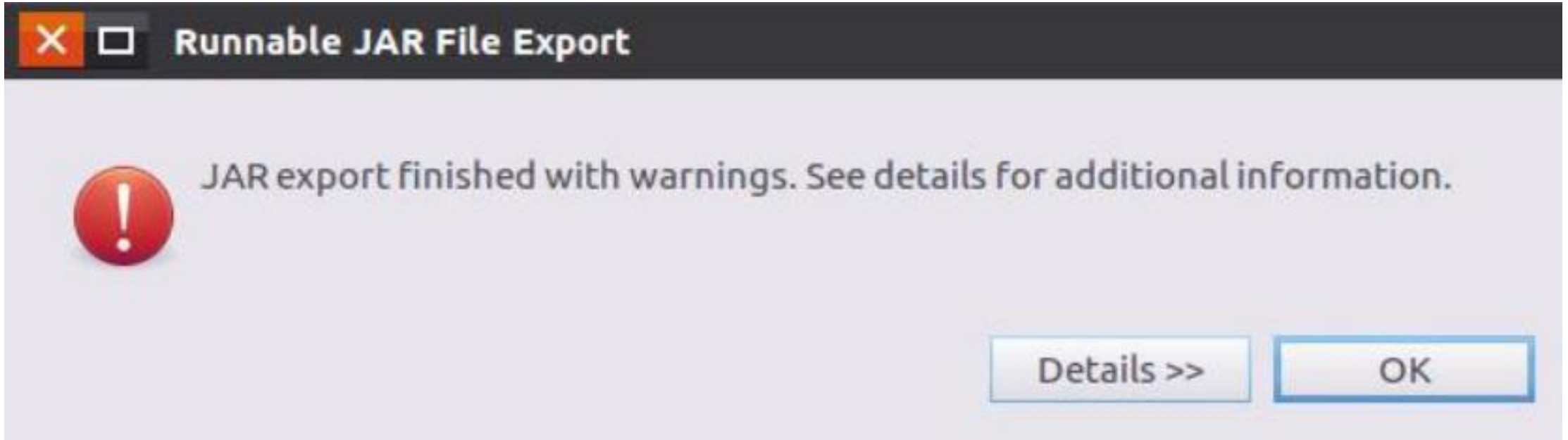
比如，这里设置为“/usr/local/hadoop/myapp/WordCount.jar”。在“Library handling”下面选择“Extract required libraries into generated JAR”。然后，点击“Finish”按钮，会出现如下图所示界面。



编译打包代码以及运行程序

■ 编译打包程序

可以忽略该界面的信息，直接点击界面右下角的“OK”按钮，启动打包过程。打包过程结束后，会出现一个警告信息界面，如下图所示。



编译打包代码以及运行程序

- 编译打包程序

可以忽略该界面的信息，直接点击界面右下角的“OK”按钮。至此，已经顺利把WordCount工程打包生成了WordCount.jar。可以到Linux系统中查看一下生成的WordCount.jar文件，可以在Linux的终端中执行如下命令：

```
cd /usr/local/hadoop/myapp
ls
```

可以看到，“/usr/local/hadoop/myapp”目录下已经存在一个WordCount.jar文件。

编译打包代码以及运行程序

- 运行程序

在运行程序之前，需要启动Hadoop，命令如下：

```
cd /usr/local/hadoop
./sbin/start-dfs.sh
```

在启动Hadoop之后，需要首先删除HDFS中与当前Linux用户hadoop对应的input和output目录（即HDFS中的“/user/hadoop/input”和“/user/hadoop/output”目录），这样确保后面程序运行不会出现问题，具体命令如下：

```
cd /usr/local/hadoop
./bin/hdfs dfs -rm -r input
./bin/hdfs dfs -rm -r output
```

编译打包代码以及运行程序

- 运行程序

然后，再在HDFS中新建与当前Linux用户hadoop对应的input目录，即“/user/hadoop/input”目录，具体命令如下：

```
cd /usr/local/hadoop
./bin/hdfs dfs -mkdir input
```

然后，把之前在Linux本地文件系统中新建的两个文件wordfile1.txt和wordfile2.txt（假设这两个文件位于“/usr/local/hadoop”目录下，并且里面包含了一些英文语句），上传到HDFS中的“/user/hadoop/input”目录下，命令如下：

```
cd /usr/local/hadoop
./bin/hdfs dfs -put ./wordfile1.txt input
./bin/hdfs dfs -put ./wordfile2.txt input
```

编译打包代码以及运行程序

- 运行程序

如果HDFS中已经存在目录“/user/hadoop/output”，则使用如下命令删除该目录：

```
cd /usr/local/hadoop
./bin/hdfs dfs -rm -r /user/hadoop/output
```

现在，就可以在Linux系统中，使用hadoop jar命令运行程序，命令如下：

```
cd /usr/local/hadoop
./bin/hadoop jar ./myapp/WordCount.jar input output
```

编译打包代码以及运行程序

- 运行程序

上面命令执行以后，当运行顺利结束时，屏幕上会显示类似如下的信息：

```
... //这里省略若干屏幕信息
2023-06-17 02:50:31,862 INFO mapred.LocalJobRunner: reduce task executor complete.
2023-06-17 02:50:32,532 INFO mapreduce.Job:  map 100% reduce 100%
2023-06-17 02:50:32,533 INFO mapreduce.Job: Job job_local51129470_0001 completed
successfully
2023-06-17 02:50:32,578 INFO mapreduce.Job: Counters: 36
... //这里省略若干屏幕信息
```

编译打包代码以及运行程序

- 运行程序

词频统计结果已经被写入了HDFS的“/user/hadoop/output”目录中，可以执行如下命令查看词频统计结果：

```
cd /usr/local/hadoop
./bin/hdfs dfs -cat output/*
```

上面命令执行后，会在屏幕上显示如下词频统计结果：

```
Hadoop 2
I 2
Spark 2
 fast 1
good 1
is 2
 love 2
```